

Nanocraft User Guide

Justin Grill

1.0.0 Release

Contents

1	Installation	2
2	Input and configuration files	2
2.1	Configuration files: what you need to edit	2
2.2	First execution of the notebook	4
2.3	Parallel execution	4
2.4	Log files	4
2.5	Exploring a large number of argument combinations: the parameter grid	4
2.6	File batching with <code>glob</code>	6
3	Main modules and functions	6
3.1	Generation of nanocrystals	6
3.1.1	Loading the supercell	6
3.1.2	Cutting the supercell	7
3.1.3	Cutting the supercell: random point	8
3.1.4	Cutting the supercell: example of parallel running	9
3.1.5	Support of non-included cutting mask: how to add you own mask	9
3.1.6	Important note regarding crystal generation	10
3.2	Facet detection	10
3.3	Substitution and vacancy defect	10
3.4	Shuffle	11
3.5	Passivation	14
3.6	Functionalization	17
3.6.1	Functionalization of capped NPs	17
3.6.2	Functionalization of bare structures	18
3.7	Basic molecular editing	19
4	Visualization and diagnostics	20
4.1	Structure visualization	20
4.2	Facet visualization	21
4.3	Tessellation visualization	21

About

This document presents the Nanocraft toolkit and its functionalities. Sect. 1 covers the installation procedure. Sect. 2 describes the configuration files required to run the program and introduces key concepts such as the parameter grid and parallel execution. Sect. 3 documents the main modules and functions, from nanocrystal (NC) generation to functionalization, substitution, and structural perturbation. Sect. 4 presents the visualization and diagnostic tools available through a dedicated Jupyter Notebook. Code segments corresponding to Jupyter Notebook cells are displayed as blue rectangles. A complete code block begins with IN [INT], while the associated output is shown in a gray rectangle starting with OUT [INT].

```
In [1]:  
# Some code, ran by executing the notebook cell.
```

```
Out [1]:
```

Correspond output printed to screen.

1 Installation

The installation of the Nanocraft toolkit is minimal: download the files from Github and follow the instructions of the `INSTALL` file. After that, open the main Jupyter notebook to execute the code. It is recommended but not mandatory to create a virtual environment for the project. All required modules are listed in the `requirement.txt` file.

You may run the examples in the `examples/` folder to verify the installation was successful.

2 Input and configuration files

2.1 Configuration files: what you need to edit

One of Nanocraft main objectives is to allow the manipulation of functionalized nanocrystals (NCs), from their generation to their functionalization. In order to detect the core of the nanoparticles (NPs) and capping ligands, to know how to attach a fragment to the crystal, or even simply to know in what order the atoms should appear in a given chemical formula, the program relies on some configuration files that are located in the `misc/` folder of the installation directory. These files can be freely edited by the user if needed. A configuration module `cnfg` is loaded on first call of the program (first cell of the Notebook) and stores the misc files data into global dictionaries that can be accessed by functions from others modules. The global dictionaries are maps between a key and a value. In order to update the global variables upon modification, the user has to restart the Notebook kernel. Naturally, the first cell of the Notebook has to be executed again.

Here is a description of the content of each of the misc files.

`atomic.dat`

This file contains all the necessary, basic information about the elements: covalent and atomic radii, electronegativity, atomic mass (a.m.u), most common oxidation state, etc. A dictionary is stored in memory for each quantity. In particular, the covalent radii and electronegativities are used to compute default bond lengths between atoms using the formula

$$r_{AB} = r_{cov,A} + r_{cov,B} - \eta|\chi_A - \chi_B|,$$

where A and B are two atomic label. By default $\eta = 0.09$.

`bonds.dat`

This file allows the user to define custom bond lengths to use instead of the default ones computed using the information in `atomic.dat`. It is also possible to modify the radius of the neighbors search method. Please inspect this file to see how to proceed.

`angles.dat`

This file allows the user to choose the grafting angle at which a fragment is attached to the NP. If the fragment admit more than one atom, an angle between the vertical axis of the local frame (defined at the surface of the NP) and the intramolecular axis can be defined. By default an angle of 104.5° is assumed. It can be changed by specifying the desired value for a set of atomic types in the file. The order of the atomic labels matters.

`order.dat`

This files contains all the chemical elements of the periodic table. The order of appearance of atomic labels in the file allows one to control the order in which elements appear in the chemical formulas.

`coordination.json`

While the user is encouraged to modify the previous files to match better its own system, it not obligatory. On the contrary, the modification of `coordination.json` is mandatory. It defines the coordination of each atoms in all the parts of the NPs, namely the core, the surface, and the ligands (capping) fragments. It is loaded to a dictionary map and used extensively throughout the program. In Fig. 1 and 2 are illustrated two examples of how to fill the coordination file. The NP is divided in the three parts aforementioned. For each part, the user must fill the corresponding list of atoms and their coordination. It is worth noting that an atom can appear in multiple categories, with various coordination numbers.

```

{
  "_comment1": "Coordination of atoms in the nanoparticle, filling this
file is highly recommended.",
  "_comment2": "Fill each section with lines formatted as: 'type':
[int_1, int_2_optional, ...]",
  "bulk_coord": {
    "Cd": [4],
    "Te": [4]
  },
  "surf_coord": {
    "Cd": [2, 3],
    "Te": [2, 3]
  },
  "ligs_coord": {
  }
}

```

Figure 1: Example of `coordination.json` file for the making of CdTe quantum dots. Only the bulk atoms in the core and at the surface are given. The atoms belonging to the surface may exhibit one or two dangling bonds with respect to atoms in the bulk.

```

{
  "_comment1": "Coordination of atoms in the nanoparticle, filling this
file is highly recommended.",
  "_comment2": "Fill each section with lines formatted as: 'type':
[int_1, int_2_optional, ...]",
  "bulk_coord": {
    "Cd": 4,
    "Te": 4
  },
  "surf_coord": {
    "Cd": [2, 3],
    "Te": [2, 3]
  },
  "ligs_coord": {
    "C": 4,
    "H": 1,
    "O": 2,
    "S": 2
  }
}

```

Figure 2: Updated `coordination.json` file with atoms belonging to the passivation shell and/or organic ligands.

2.2 First execution of the notebook

Nanocraft functionalities are accessed through a Jupyter Notebook interface. It requires no coding. There is no need to learn the module functions in order to use it. Executing the code in some python script remains of course possible if you want to integrate it into your workflow.

There is no proper input file needed to run the code: once all the configuration files are set up for your system (in particular, `coordination.json`) simply reboot the Jupyter Notebook kernel and execute the cells corresponding to what you want to do.

In all the examples below, it will be assumed that you first ran a cell to set up the Nanocraft environment (see Sect. 2.1), it should contains at least the following elements, assuming each function is loaded when needed. You may modify the cell to add your systematic import such as `import numpy as np`, etc.

```
In [1]:
import sys
from src.nanocraft.cnfg import *           # global variables
from src.nanocraft.log_cnfg import *      # log file
from src.nanocraft.utls import grid, run_parallel
from tqdm import tqdm # progress bar, optional
```

```
Out [1]:
Initializing nanocraft configuration...
Done! If any changes are made to the nanocraft misc files, please restart the
environment.
```

2.3 Parallel execution

Nanocraft allows parallelization through the `multiprocessing` Python module. However, only a limited set of functions can be ran in parallel. Are concerned the functions that accept in argument a *dictionary of settings*, generated from a *settings grid* with the function *grid*. The latter terms in italic are key component of the Nanocraft toolkit and are detailed in Sect. 2.5. You may want to first read this section.

In order to run a function across multiple processors, replace the FOR loop on the settings grid elements by a single call to the `parallel_run` function. This wrapper function takes in argument the function you want to run in parallel, the whole settings grid, the number of workers (CPUs), and all the remaining arguments of the function to be ran omitting the settings dictionary. An example of how to run the code in parallel is given in Sect. 3.1

2.4 Log files

Nanocraft uses the external library `loguru` as its logging system. When the first cell of the Jupyter is executed (import of `log_cnfg` and `cnfg`), a logger object is created which can be accessed by other modules to display messages with several levels of gravity: debugging, information, warning and error. The warning are usually thrown when the program encountered some missing value in the input or did not function properly but could still figure out a solution (default value, ignoring the instruction). When the program is terminated by some error in the terminal, it is also logged. An history of 3 log files are kept, the older are automatically removed.

2.5 Exploring a large number of argument combinations: the parameter grid

In order to explore a large number of possible parameter values, whether it is cut radii for the NC generation or cutting shapes, densities of ligands for the functionalization or ligands species, a given function must be called over all possible combinations of parameters. In order to solve the struggle and lack of generality to let the user write its own set of nested FOR loops, which may be numerous, the main functions of Nanocraft takes in argument a dictionary of settings, whose keys are the function argument, and whose values are naturally the value of the corresponding argument.

Hence, only a single loop over all possible settings dictionaries is needed instead of multiple nested loops. In practice, the user is prompted to fill a dictionary whose keys are identical to the setting dictionary taken by the function. Instead of assigning a single value to a key, the user can specify a list of values he wants to try. A settings grid, containing all possible combinations of parameters (=keys) values is then constructed using a dedicated function `grid`.

The settings grid is nothing else than a Cartesian product between all parameters values from the initial dictionary and is stored as a list of individual settings dictionaries. To illustrate the settings grid formalism, let

us consider the examples below. In the first block, `settings` has 3 keys corresponding to a given cut radius, a cutting shape and an acceptance criterion for the obtained structure. For each key, a list of values is given. If a unique value is given, it will be treated internally as a singleton. It is worth noting already that it is also possible to assign nested lists to a given key. This is particularly relevant when performing random displacement (see Sect. 3.4) or passivation (see Sect. 3.5). In this example, the Numpy's `arange` function is used to generate a list (array format) of increasing float with step 0.5, between 0.5 and 2.01. You may want to use `arange` or `linspace` as well.

```
In[1]:
import pprint
import numpy as np
settings = {
    'radius': np.arange(0.5, 2.01, 0.5),
    'shape': ['sphere', 'cube'],
    'criterion': 'auto'
}
settings_grid = grid(settings)
pprint.pprint(settings_grid)
```

```
Out[1]:
[{'criterion': 'auto', 'radius': 0.5, 'shape': 'sphere'},
 {'criterion': 'auto', 'radius': 0.5, 'shape': 'cube'},
 {'criterion': 'auto', 'radius': 1.0, 'shape': 'sphere'},
 {'criterion': 'auto', 'radius': 1.0, 'shape': 'cube'},
 {'criterion': 'auto', 'radius': 1.5, 'shape': 'sphere'},
 {'criterion': 'auto', 'radius': 1.5, 'shape': 'cube'},
 {'criterion': 'auto', 'radius': 2.0, 'shape': 'sphere'},
 {'criterion': 'auto', 'radius': 2.0, 'shape': 'cube'}]
```

The `settings` dictionary is converted in a grid *ie* a list of parameter dictionaries that can be given to the function we want to call. In the example below is presented the basic structure of functions accepting a settings dictionary as argument. The first cell shows a `test` function which perform some operation based on the value of the `radius`, `shape` and `criterion` parameters. To parse all of the desired values we thus have to write three loops. Instead, we illustrate in the second cell a `test` function that takes in argument a dictionary. It can thus be called successively using a single loop and be evaluated on all possible combination of parameters.

```
In[1]:
def test(r, s, c):
    output = # some operation using r, s, c
    return output

for radius in np.arange(0.5, 2.01, 0.5):
    for shape in ['sphere', 'cube']:
        for criterion in ['auto']: # the number of loop may go on...
            test(radius, shape, criterion)
```

```
In[1]:
def test(settings):
    r, s, c = settings['radius'], settings['shape'], settings['criterion']
    output = # some operation using r, s, c
    return output

settings = {
    'radius': np.arange(0.5, 2.01, 0.5),
    'shape': ['sphere', 'cube'],
    'criterion': 'auto'
}

settings_grid = grid(settings) # generates all the combinations

for settings in settings_grid: # a single loop
    test(settings)
```

A progress bar can be added to the unique FOR loop using the `tqdm` package imported in the first cell. The two last lines of the previous block become:

```
for settings in tqdm(settings_grid, desc="Progress"):
    test(settings)
```

The `grid` function admits two optional arguments: `duplicate` and `collapse`. The argument `duplicate` is an integer which allows to repeat the full grid a corresponding number of time. It is particularly useful when performing operations involving randomness, such as the positioning of the ligands, or some random displacements. It allows to generate statistical samples. The second optional keyword `collapse` allows to remove from the parameter grid invalid or useless combination of parameters. A tuple is given containing a parameter to check, a parameter to collapse (*ie* skip) and the values of the first parameter for which the second should be collapsed. An illustration of this functionality is given in Sect. 3.1. The `grid` function is part of the `utls` utility module containing essential tools for parameters grid management, path handling, vector/matrix operations and parallelization.

2.6 File batching with glob

As stressed in Sect. 2.5, the purpose of Nanocraft is to ease the repetitive manipulations that one can do to a set of structures. It naturally follows that the input structures (XYZ format) should not be specified by hand. Most importantly they should be provided to functions via a key in the settings grid. A useful tool to automatically construct a list of files from the XYZ files present in a directory is the Python `glob` package. It can be used along with the `os` or `pathlib` packages to get all the files matching a naming pattern.

```
In[1]:
import os
import glob
inputfiles = glob.glob(os.path.join("/path/to/xyz/files", "*.xyz"))
# Matches all .xyz files in directory "/path/to/xyz/files"
# os.path.join handles paths formatting
```

3 Main modules and functions

In this section are presented the main functions and modules of the Nanocraft toolkit. This presentation is limited to the functions highlighted in the main Notebook. As this toolkit is open-source, the user has the liberty to modify, extend, and reproduce the code.

Throughout this section, two distinct terms are used: a nanocrystal (NC) refers to a structure whose core exhibits bulk-like crystalline order, typically obtained by cutting a supercell, whereas a nanoparticle (NP) is a broader term that includes NCs as well as structures with amorphous cores, hollow and endohedral cages which are typically obtained with external methods. Most functions can process bare, capped (with passivation fragments), or functionalized (with arbitrary ligands) structures, relying on atomic coordinations read from the `coordination.json` file and on the generic neighbor search implemented in the `get_core_atoms_and_outer_layer` method from the `srflyrs` module. If the system composition is modified (for example after functionalization), this file must be updated and the notebook kernel restarted. The scope of each function, in terms of applicable structures, is explicitly recalled at the beginning of the corresponding sections.

Except for the two functions for ligand pre-processing (Sect. 3.7) and the function for loading the supercell (Sect. 3.1), all others take identical arguments: a *settings* dictionary and an output directory. The latter is optional. Default output directories are created in Nanocraft installation directory.

Diagnostic and visualization methods are discussed in Sect. 4; they may be useful for quickly verifying the integrity of your structures at a glance.

3.1 Generation of nanocrystals

3.1.1 Loading the supercell

The generation method presented in this subsection is limited to bare NCs only. It produces polyhedral NC cores as a three-step process:

1. The supercell is loaded/generated using `load_supercell` (`sprcll` module);
2. The supercell is centered at some given site using `apply_supercell_transform` (`sprcll` module);

3. Cutting masks are applied and the obtained structures are eventually saved using `generate` (`ncgen` module).

The function `load_supercell` accepts either a supercell in the XYZ format prepared externally (*eg*, VESTA) or a CIF file. The latter is converted to a supercell of desired size using Python `pymatgen` module. The asymmetric unit cell is read and a correct conventional cell is created using point group operations. The size of the supercell is controlled through the parameter `supercell_size` which is a list of three integers. To load the supercell, run the following code block.

```
In[1]:
from src.nanocraft.sprcll import load_supercell
supercell = load_supercell(inputfile="1540817.cif", supercell_size=[9,9,9])
```

```
Out[1]:
Supercell loaded successfully from 1540817.cif.
FYI, this supercell can fit a sphere of radius: 43.486 Å.
Choose you parameter accordingly.

Snapshot of 20 atoms closest to the supercell's COM:
array(['Te', -0.78500000000005792, -0.78500000000003341,
      ...,
      ['Cd', -2.3550000000000576, -2.3550000000000331, 3.924999999997094]],
      dtype=object)
```

Upon execution, the code computes the radius of the biggest sphere that can fit inside the supercell. This can be used as estimation to chose a higher bound to the cutting radius for the generation. It can happen that the program fails to read the data in the CIF file. In that case, it is recommend to try another one or to try repairing it. To this end, you may inspect the formatting of the provided tests CIF files.

3.1.2 Cutting the supercell

Once the supercell is loaded, the function `generate` can be called to generate polyhedral NC cores by cutting along geometric masks and applying a coordination-based acceptance criterion. The second step listed in Sect. 3.1.1, *ie* the centering of the supercell on a specific site, is actually performed by the cutting function. Because the user can decide to provide a list of central sites to test thanks to a corresponding key in the settings dictionary, `generate` calls `apply_supercell_transform` on its own. Here is a description of the two functions.

`generate(settings, supercell, output_dir=None, remove_duplicates=True)`

Main generation function. `supercell` is the output of `load_supercell`, `output_dir` is optional. `settings` is a dictionary which accepts the following keys/values:

`rcut`: float, required.

Cutoff radius (nm) for mask extraction (converted to Å internally).

`fratio`: float, required.

Ratio $F = R_{111}/R_{100}$ for truncated octahedron mask (can be ignored for sphere/cube with the `collapse` keyword for the settings grid). It can also be used to define an additional cutting distance for your own masks.

`mask_shape`: str, required.

Polyhedral mask: "sphere", "cube", "octahedron", etc. See the `ncgen` module for all implemented masks.

`central_site`: str, list[str], list[float], required.

Central atom label(s) or cartesian coordinates of a point. A random point can also be chosen with the value "random". The keyword `None` tells the code to ignore centering the supercell on a specific site, and leave it to default position which is the global center of mass. A list of valid values is given in the example below.

`acceptance_criterion`: str, optional (default "auto").

Acceptance threshold of the generated structures. "auto": iteratively remove undercoordinated atom until valid, "strict": reject if there is any undercoordinated atoms, "tolerant": stricter version "auto". For "auto" and "tolerant", atoms with insufficient coordination are removed. The removal step is repeated three times only for "tolerant" and the structure is saved even if they are remaining uncoordinated atoms. Use "none" to avoid any check and force saving.

```
apply_supercell_transform(supercell, central_site)
```

Shift the center of mass of the supercell to `central_site`, provided in same format as in `generate`.

In the following example, three different cutting masks will be tried for a large number of possible cutting radii. Structures with various centering sites will also be generated. The nested list `['Cd', 'Te']` given in `central_site` allows to center the supercell at the middle point of two closest 'Cd' and 'Te'. Please note that a tuple of size 2 *ie* ('Cd', 'Te') works the same. It is worth noting that any subsystem of atoms can be given. All cut clusters will be iteratively checked for atoms with wrong coordination until their surface coordination is valid.

```
In [1]:
from scr.nanocraft.ncgen import generate

output_dir = "./tmp"

settings = {
    'rcut': np.arange(0.4, 2.5, 0.05), # nm
    'fratio': np.arange(1.0, 2.9, 0.05),
    'mask_shape': ['cube', 'sphere', 'octahedron'],
    'central_site': ['Cd', 'Te', ['Cd', 'Te']],
    'acceptance_criterion': 'auto'
}

settings_grid = grid(settings, collapse=('mask_shape', 'fratio', ['sphere',
    'cube']))

for settings in tqdm(settings_grid, desc="Progress"):
    generate(settings=settings, supercell=supercell, output_dir=output_dir)
```

In this example, multiple values for the second cutting radius controlled through `fratio` are provided. Since this parameter is only relevant for the octahedron mask, it will lead to equivalent entries of the settings grid when dealing with other masks. To circumvent this, an elegant solution is to make use of the `collapse` keyword of `grid`.

`collapse:` tuple, optional.

A tuple of size 3 where the first and second elements are keys (`str`) of the settings dictionary and third element is a list of valid values of the first key. `collapse=(key1, key2, [val1, val2, val3])` should be interpreted as follows: when `KEY1` takes any of the values `VAL1`, `VAL2` or `VAL3` then only one value of amongst all possible values of `KEY2` is retained.

This keyword ensures that only a single value of a given parameter is visited when another parameter takes specific values. As a result, there are no longer equivalent entries in the parameter grid.

3.1.3 Cutting the supercell: random point

It is possible to specify the Cartesian coordinates of a point to be used as the center of mass of the supercell, *eg* `central_site: [0.0, 0.0, 0.0]` which is equivalent to `central_site: None`. The center of the supercell can be chosen to be random using `central_site="random"`. It will result in a global translation of all atoms by a random vector in a sphere of 10 Å. Since this translation is effectuated for each entry of the settings grid, all resulting structures will all be centered at different random points. In order to chose once and for all a fixed (yet random) point, you can use the following trick:

```
In [1]:
supercell = load_supercell(inputfile="1540817.cif", supercell_size=[9,9,9])

random_supercell = apply_supercell_transform(supercell, 'random')

settings = {
    ...
    'central_site': None, # skip the call to apply_supercell_transform in
        generate
    ...
}
```

```

settings_grid = grid(settings, ...)

for settings in tqdm(settings_grid, desc="Progress"):
    generate(settings=settings, supercell=random_supercell)

```

3.1.4 Cutting the supercell: example of parallel running

The FOR loop used for the generation of structures may take a lot of time to run, especially when the size of the settings grid is large. One source of the program's slowness is linked to the initialized supercell. It is centered at each call of the `generate` function even if a unique central site is selected. For supercell with tens of thousands of atoms it might represent a millisecond operation. The true culprit of the program's slowness is the acceptance criterion checking step. The "auto" keyword allows to only save candidate structures with the right surface coordination, but this requires multiple heavy neighbors searches for each atom of each structures. For this reason, a parallel search of structures is recommended. It requires very minimal modification to the previous illustrations. The FOR loop calling the `generate` function is replaced by a single call to the `run_parallel` function.

`run_parallel(function, grid, n_workers, **kwargs)`

Wrapper used to run function in parallel on all possible combinations of parameter of `grid`, using `n_workers` CPUs. All arguments of `function`, omitting its first argument `settings` dictionary, are passed in `**kwargs`.

The following code blocks illustrate how to run `generate` on eight cores in parallel. The first cell is the usual call up to now while the second one make use of parallelization.

```

# construction of settings_grid, skipped here

# loop over all settings dictionaries of settings_grid
for settings in settings_grid:
    generate(settings, supercell, output_dir)

```

```

# construction of settings_grid, skipped here

# single call to run_parallel, which takes the full settings_grid
run_parallel(generate, settings_grid, 8, supercell=supercell, output_dir=
    output_dir)

```

3.1.5 Support of non-included cutting mask: how to add you own mask

Although Nanocraft only comes with a few available cutting shapes, it is fairly simple to add you own. Adding new masks will allow you to generate NCs with more complex shapes whether is it by applying a single cutting mask to the supercell, or by successively applying cutting masks. One could for example add a mask consisting of only two planes in opposite direction enabling the generation of nanowires / slabs from a given supercell. The cutting radius `rcut` would define the distance from the center of mass to the cutting plane.

In order to add new shapes, the user should edit the `ncgen` module and follow the steps described here.

1. Define a function that cuts the desired shape from the bulk data (see `cut_tetrahedron_from_bulk`).
2. Add the function to the `accepted_mask_shapes` dictionary in `generate`. A label corresponding to the new cutting method has be chosen in order to select it in the settings grid.
3. If the shape has facets, you can set the value of the `save_facets` key to True and eventually modify the facet detection method (see Sect. 3.2) function `generate_facets`.

Each entry of `accepted_mask_shapes` dictionary is formatted as follow:

```

accepted_mask_shapes = {
    "shape1": {
        "args": [rcut], # list of argument to pass to this cutting function
        "cut_shape_from_bulk": func_name_shape1, # function associated to shape1
        "save_facets": False # whether or not facet generation is requested
    },

```

```
"shape2": {
    ...
},
...
}
```

3.1.6 Important note regarding crystal generation

One important note is that the shape of the chosen cutting mask will not necessarily match that of the resulting NC. Indeed, when using the `auto` or `tolerant` acceptance criteria, atoms will be successively removed from the structure until the nearest valid surface structure is reached. It therefore follows that a given mask may lead to NPs also generated by a different mask. Thus, although the program saves only unique structures by default within a specific folder at generation time, identical structures may still appear across multiple folders. One should therefore not assume that a folder whose name starts with, for example, `"sphere"` or `"cube"` will contain only spherical or cubic structures. In order to identify the actual shape of NPs (for instance, with the goal of labeling them and studying shape-dependent property differences), it is necessary to visualize them. For this purpose, a dedicated notebook is provided, which includes several visualization methods, notably one that renders the highest-order enclosing polygon of the structure. Furthermore, to identify duplicate structures across the various folders named after `mask_shape` and `central_site`, a separate notebook is provided that reads all structures and determines which ones are unique. The comparison is performed first on the number of atoms, and for configurations sharing the same atom count, the surface area is computed to determine whether they represent the same structure existing in two different forms or not.

3.2 Facet detection

The `facts` module handles facet recognition and identification of Miller indices, and is applicable to bare NCs typically generated with method in Sect. 3.1. In principle the user does not need to call himself any of the functions of this module. It intervenes either when the NPs are generated or when facet specific operations are asked. On generation of a given structure, the information on its facets are saved in two files (see how the facet detection algorithm works) in the folder 'facets', created at the same location than the structure. This information is read when trying to perform facet specific operations such as the passivation. If the XYZ file of the structure has been moved too far from its initial directory, the facets folders may not be found. In that case, the facet detection will be performed again automatically; the user needs not to do it manually. Nevertheless if the user desires to it anyway he just has to call the function `generate_facets` that takes two arguments: either the XYZ data of the bare cluster or the path to it, and the `output_dir` where should the output files be saved. By default, they will be saved at the same location than the given file. For the moment, only the bare cluster can be given (consistent with the scope of indented use case). Please note that the code makes no difference between the 3,3,3 or the 1,1,1 reticular planes, but is able to distinguish between the 1,1,1 and 1,1,-1 for instance. If the user needs to use the facet recognition method for more complex crystal cuts, it can easily adapt this program by extending the lists of facets in the function `get_facets_from_family`. The plane scanning function uses the h,k,l integer values and should work fine.

3.3 Substitution and vacancy defect

The `srflyrs` module provides tools to substitute one atomic species for another within NP models, enabling the construction of doped, alloyed, or defect-containing structures for ensemble studies. This function is applicable to any type of NP, bare, partially or fully functionalized. Atoms of type `atom_to_replace` are replaced by `replacement_atom` according to a target ratio and a specified spatial distribution. Note that setting `replacement_atom` to an empty string allows the creation of vacancies. All substitution functionality wraps into the single function `substitute`, configured through a settings dictionary. The substitution ratio η is defined as,

$$\eta(\%) = \frac{N(\text{atom_to_replace})}{N(\text{atom_to_replace}) + N(\text{replacement_atom})} \times 100.$$

`substitute(settings, output_dir=None)`

Main function for the substitution.

`nanocrystal_file`: str, required.

Path to input XYZ nanocrystal.

`atom_to_replace`: str, required.

Species to remove (e.g., "Te").

replacement_atom: str, required.
Species to insert (e.g., "S") in lieu of **atom_to_replace**.

mixing_ratio: float, required.
Target ratio of **atom_to_replace** amongst **atom_to_replace** + **replacement_atom**.

method: str, required.
Substitution strategy. Either **random** or **layer**. The former correspond to uniform random selection from all eligible atoms and latter corresponds to concentric (onion-peel) layers from the NP surface inward (see **direction**).

direction: str, optional (default "outside_in").
Layer progression: "outside_in" (surface first) or "inside_out" (core first). Applies only to "layer" method.

random_seed: bool or int, *optional* (default True)
Reproducibility: **int** (fixed seed), **True** (random), **False** (default seed is used).

The substitution function is applicable to any type of NP, with or without ligands. However, when **method="random"**, all atoms whose type matches **atom_to_replace** are considered, including those present in the ligand shell. When **method="layer"**, successive layers are built by iteratively detecting the surface of the NP core (via the "nc" keyword in **get_core_atoms_and_outer_layer**), based on the atom types and associated coordinations specified in **coordination.json**. In this case, ligand atoms are in principle excluded from the substitution.

```
In[1]:
output_dir = "./tmp"
nrep = 3 # statistical sampling

settings = {
    'nanocrystal_file': '.nanocrystals/Cd65Te65S34H68.xyz',
    'atom_to_replace': 'Te',
    'replacement_atom': 'S',
    'mixing_ratio': np.arange(0.0, 101.0, 10.0),
    'method': "layer",
    'direction': "outside_in",
    'random_seed': True
}

settings_grid = grid(settings, duplicate=nrep)

for settings in tqdm(settings_grid, desc="Progress"):
    substitute(settings=settings, output_dir=output_dir)
```

In this example, Te atoms are progressively replaced by sulfur atoms. A ratio of 0% will correspond to a CdS NP, and $\eta = 100\%$ to a pure CdTe pure NP. The initial ratio is of 66 %. The substitution Te→S is done from outside layers toward the center. The **srflyrs** module also contains all the functions related to fragment and chain detection, the identification of bulk and ligand regions, and the tessellation methods.

3.4 Shuffle

The **shffl** module provides tools to introduce controlled structural perturbations into NC models, focusing on random constrained perturbations of atomic positions and bond lengths. This function is applicable to any type of NP, from bare NCs to functionalized NPs. It allows to explore the space of configurations while preserving basic chemical properties such as coordination and bond types. Atomic positions can be randomly displaced within a specified maximum displacement, while bond perturbations are applied along the bond axis with displacements distributed according to atomic masses, meaning lighter atoms will move more than heavier ones. The module supports targeting specific regions of a NC, whether the bulk, the surface, the ligands, or the whole system, and offers control over both the fraction of atoms or bonds that are perturbed and the magnitude of those perturbations.

All shuffle functionalities are wrapped in a single function, **shuffle**, configured through a settings dictionary. A list can be assigned to a dictionary key in order to try multiple combinations of parameters. It is worth

noting that multiple subsets of atoms in the NC can be shuffled within a single function call, in which case nested lists must be used. Several examples are provided below.

shuffle(settings, output_dir=None

Main function for performing random displacement of atoms or atomic bonds.

xyz_file: str, required.

Path to the input XYZ file containing the nanocrystal.

layer: str, required.

Specifies which region of the NC to consider when selecting atoms/bonds to shuffle. Valid values for each entry are:

"bulk": bulk-like region (core) as determined from coordination maps.

"nc": outermost bulk atoms (NC surface).

"cap": ligand / capping layer.

"mix": combination of "nc" and "cap" (crystal surface plus ligands).

"all": all atoms in the structure.

atoms_bonds_set: str, tuple[str, str], required.

Defines what will be shuffled: a type of atom (str) or bond (tuple[str, str]).

percent_shuffle: float, required.

Fraction of the *available* atoms/bonds (within the specified layer) that will actually be shuffled. Floats between 0 and 1 (fraction) or between 1 and 100 (percentage) are both accepted.

max_disp: float, required.

Maximum displacement (radius r_{max}) used when shuffling atoms or bonds. Position of selected atoms is changed by a random vector with magnitude up to **max_disp**. When shuffling bonds, the code applies a mass-weighted bias to distribute the total bond length perturbation Δ_{tot} between the two atoms. The perturbation can extend or shrink bonds by drawing a random number in $r_{max} \times \mathcal{U}([-1, +1])$. The atoms (1) and (2) involved in the bond are respectively displaced along the axis defined as

$$\hat{\mathbf{n}} = \frac{\mathbf{r}_2 - \mathbf{r}_1}{|\mathbf{r}_2 - \mathbf{r}_1|},$$

by a quantity

$$\begin{cases} \Delta_1 = \left(\frac{m_2}{m_1 + m_2} + \lambda_1 \right) \Delta_{tot} \\ \Delta_2 = \left(\frac{m_1}{m_1 + m_2} + \lambda_2 \right) \Delta_{tot} \end{cases},$$

where m_1 , m_2 are atomic masses and $\lambda_{1,2}$ are optional biases. The shuffled position are

$$\begin{cases} \mathbf{r}'_1 = \mathbf{r}_1 - \Delta_1 \cdot \hat{\mathbf{n}} \\ \mathbf{r}'_2 = \mathbf{r}_2 + \Delta_2 \cdot \hat{\mathbf{n}} \end{cases}.$$

The case $\Delta_{tot} > 0$ corresponds to bond stretching : the two atoms move away from previous positions in opposite direction; $\Delta_{tot} < 0$ corresponds to shrinking of the bond: each atom moves closer to the other. The biases $\lambda_1 = -\lambda_2$ activate if $|m_1 - m_2| > 4$ (a.m.u). If this condition is verified,

$$\lambda_1 = \text{sign}(m_2 - m_1) \min(\kappa, \kappa \mu_{inv}),$$

with $\kappa = 0.2$ and reduced mass relative to O-H bond reference. This additional bias is added when the two atom involved in the bond have very different masses. For very dissimilar masses (e.g., Cd-H), the unbiased displacement $\Delta_1 = \frac{m_2}{m_1 + m_2} \approx 0$ would pin the heavy atom. Setting $\lim_{m_1 \gg m_2} \Delta_1 = \lambda_1$ ensures minimum motion for both atoms, with a floor value of $\lambda_{1,2} \Delta_{tot}$ scaled by reduced mass to avoid structural inversion (light atom tunneling through heavy one). By default $\kappa = 0.2$. Both κ , μ_{inv} and the threshold value of 4, can be changed in the module.

random_seed: bool or int, optional (default=True).

Controls reproducibility of the random shuffling. If an integer is provided, it is used as the seed. If **random_seed** is True, a random seed is generated. If **random_seed** is False, a default seed is used.

```
In [1]:
from src.nanocraft.shffle import shuffle
```

```

output_dir = "../tmp"
nrep = 5

settings = {
    'xyz_file': ['structure.xyz'],
    'layer': ["bulk", "bulk", "cap"],
    'atoms_bonds_set': [['Cd', 'Te', ('S', 'H')]],
    'percent_shuffle': [[0.35, 0.35, 0.4]],
    'max_disp': [[0.6, 0.6, 0.4]],
    'random_seed': True
}

settings_grid = grid(settings, duplicate=nrep)

for settings in tqdm(settings_grid, desc="Progress"):
    shuffle(settings=settings, output_dir=output_dir)

```

This code above shuffles successively the cadmium, tellurium and thiol S-H bonds belonging respectively to the bulk and surface of the `structure.xyz`. In total, 35% of all Cd and Te belonging to the crystal core will be displaced in a sphere of radius 0.6 Å while 40% the S-H bonds will be stretched by 0.4 Å. The parameter `nrep` allows to reproduce the procedure 5 times. Five files are produced in `output_dir`, named after `xyz_file` with a suffix from 1 to 5.

The parameters `layer`, `percent_shuffle`, `max_disp` must be of same size as `atoms_bonds_set`. If the latter is a nested list of 3 elements, the former must be nested lists of three elements as well. Otherwise, their first element will be duplicated to match the correct size. The user must be very careful when using the nested list since `atoms_bonds_set`: `["Cd", "Te", ("S", "H")]` would shuffle exclusively a type of atom or bond and not the three successively. The nested list `['Cd', 'Te']` shuffles both types *successively* in a sphere of radius 'max_disp' while the nested tuple `(('Cd', 'Te'))` allows to target corresponding bond for shuffling. Let's consider the following example.

```

In[1]:
settings = {
    "xyz_file": "structure.xyz",
    "layer": ["bulk", "nc"],
    "atoms_bonds_set": [["Cd", "Se", ("Cd", "Se")]],
    "percent_shuffle": 0.4,
    "max_disp": [0.3, 0.1],
    "random_seed": True,
}

```

In this example, `atoms_bonds_set` is a nested list containing one element which is a list of size 3. The output of `grid(settings)`, namely the full settings grid will consist of the four possible combinations:

```

[{'atoms_bonds_set': ['Cd', 'Te', ('Cd', 'Te')],
  'layer': 'bulk',
  'max_disp': 0.3,
  'percent_shuffle': 0.4,
  'random_seed': True,
  'xyz_file': 'structure.xyz'},
 {'atoms_bonds_set': ['Cd', 'Te', ('Cd', 'Te')],
  'layer': 'bulk',
  'max_disp': 0.1,
  'percent_shuffle': 0.4,
  'random_seed': True,
  'xyz_file': 'structure.xyz'},
 {'atoms_bonds_set': ['Cd', 'Te', ('Cd', 'Te')],
  'layer': 'nc',
  'max_disp': 0.3,
  'percent_shuffle': 0.4,
  'random_seed': True,
  'xyz_file': 'structure.xyz'},
 {'atoms_bonds_set': ['Cd', 'Te', ('Cd', 'Te')],
  'layer': 'nc',
  'max_disp': 0.1,

```

```
'percent_shuffle': 0.4,
'random_seed': True,
'xyz_file': 'structure.xyz']}]
```

When passed to the `shuffle` function, each of this setting dictionary will be transformed into the following dictionaries

```
[{'atoms_bonds_set': ['Cd', 'Te', ('Cd', 'Te')],
'layer': ['bulk', 'bulk', 'bulk'],
'max_disp': [0.3, 0.3, 0.3],
'percent_shuffle': [0.4, 0.4, 0.4],
'random_seed': True,
'xyz_file': 'structure.xyz'},
{'atoms_bonds_set': ['Cd', 'Te', ('Cd', 'Te')],
'layer': ['bulk', 'bulk', 'bulk'],
'max_disp': [0.1, 0.1, 0.1],
'percent_shuffle': [0.4, 0.4, 0.4],
'random_seed': True,
'xyz_file': 'structure.xyz'},
{'atoms_bonds_set': ['Cd', 'Te', ('Cd', 'Te')],
'layer': ['nc', 'nc', 'nc'],
'max_disp': [0.3, 0.3, 0.3],
'percent_shuffle': [0.4, 0.4, 0.4],
'random_seed': True,
'xyz_file': 'structure.xyz'},
{'atoms_bonds_set': ['Cd', 'Te', ('Cd', 'Te')],
'layer': ['nc', 'nc', 'nc'],
'max_disp': [0.1, 0.1, 0.1],
'percent_shuffle': [0.4, 0.4, 0.4],
'random_seed': True,
'xyz_file': 'structure.xyz'}]
```

It is worth noting that the following input would have produced a single valid combination of parameters.

```
In[1]:
settings = {
    "xyz_file": "structure.xyz",
    "layer": [["bulk", "nc"]],
    "atoms_bonds_set": [["Cd", "Te", ("Cd", "Te")]],
    "percent_shuffle": 0.4,
    "max_disp": [[0.3, 0.1]],
    "random_seed": True,
}
```

```
[{'atoms_bonds_set': ['Cd', 'Te', ('Cd', 'Te')],
'layer': ['bulk', 'bulk'],
'max_disp': [0.3, 0.3, 0.3],
'percent_shuffle': [0.4, 0.4, 0.4],
'random_seed': True,
'xyz_file': 'structure.xyz'}]
```

3.5 Passivation

The `pssv` module handles the passivation of NC surfaces by attaching molecular fragments to selected anchor sites, in bare NCs only. It supports on-top and bridge binding modes, facet and coordination constraints for the selection of anchor atom subsets, and automatic fragment removal to achieve charge balance.

The entire passivation procedure is controlled by the `passivate` function, configured through a settings dictionary. As with `shuffle`, nested lists enable multi-step passivation schemes within a single function call, though successive independent executions of the program are equally supported. It is worth noting that the procedure is designed to produce a closed passivation shell around the cluster, and will automatically remove excess fragments if too many were added. The procedure is currently limited to molecular fragments containing between 1 and 3 atoms, which serve as templates for the functionalization.

Contrarily to the `shuffle` and `substitute` functions that can be called on any NPs with or without ligands, `passivate` is limited to bare NCs as generated with `generate`. Several methods are implemented to control both the way the oxidation state is computed and the manner in which passivating groups are removed according to their charge.

`passivate(settings, output_dir=None)`

Main function for the passivation of dangling bonds in bare NC.

`nanocrystal_file`: str, required.

Input nanocrystal XYZ path.

`passiv_scheme`: tuple[str, str], required.

One step of the passivation scheme. Each tuple has the format (`anchor_atom`, `fragment_formula`), where the anchor atom belongs to the structure and the the fragment formula is a str containg up to 3 atoms.

`binding_site`: str, required

Binding mode of the fragment to the anchor. "ontop" (single atom) or "bridge" (atom pair). In case were `passiv_scheme` is given as a nested list, a nested list of the same length as to be given as well.

`facet_constrain`: str, optional (default 'all')

This keyword allows passivation to be restricted to a single facet family. Accepted values are "all", "100", "110" and "111". The list must have the same length has `passiv_scheme`, otherwise the first given element in duplicated to match the right size. The constrain on coordination has priority over the facet constrain.

`coord_constrain`: str or int, optional (default 'all')

This keyword allows passivation to be restricted to specific coordination numbers for the selected atom. Accepted values are 'all' or an integer. The list must have the same length has `passiv_scheme`, otherwise the first given element in duplicated to match the right size.

`neutral_reach`: str, optional (default "full")

Control how passivation fragments are removed in case the final oxidation degree is not zero. Choosing "full" means that no group will be removing and the shell will remain intact. Choosing "random" or "pseudorandom" will cause fragment removal. If "pseudorandom" or "random" are chosen, fragments with the highest oxidation state of opposite sign to current NC charge will be removed. If "pseudorandom", the fragments are removed from the atom with the greatest coordination (smaller number of dangling bonds first).

`oxidation_method`: str, optional (default "all_ox")

Control how the total oxidation state of the passivated cluster is computed. Accepted values are: "all_ox": the total oxidation state is computed as the sum of the most common oxidation state for all atoms. The value corresponding to each atom is read from the `atomic.dat` file.

"surface_ox": only the atoms belonging to the surface of the NC (outer layer of the bulk + capping fragments) at taken into account in the sum described above.

"surface_mix": only the atoms belonging to the surface of the NC (outer layer of the bulk + capping fragments) at taken into account as in "surface_ox". The oxidation state for the atoms of the bulk is read from `atomic.dat` while the values for the capping fragments are read from `coordination.json`.

"None": force the oxidation degree to be set to zero, so that no passivating group is removed, behaving exactly like `neutral_reach="full"`. It is intended primarily for debugging purposes, as it prevents fragment removal and writes to the logger the oxidation degree computed by each of the three other methods.

`random_seed`: bool/int, optional (default True)

Random seed control.

A typical call to the passivation function is done as follows:

```
In[1]:
from src.nanocraft.pssv import passivate

output_dir = "./tmp"
nrep = 1

inputfiles = glob.glob(os.path.join("/path/to/xyz_start", "*.xyz"))

settings = {
```

```

'xyz_file': inputfiles,
'passiv_scheme': [['Au', 'SiO2']],
'binding_site': ['ontop'],
'facet_constrain': ['all'],
'coord_constrain': ['all'],
'neutral_reach': 'full'
}

settings_grid = grid(settings, duplicate=nrep)

for settings in tqdm(settings_grid, desc="Progress"):
    passivate(settings=settings, output_dir=output_dir)

```

In this example, all gold atoms belonging to the surface will be capped with SiO₂, no matter their facet or coordination (at least one dangling bond is required in all cases). It is worth noting that the opening angle of this fragment can be chosen by editing the `angles.dat` misc file. By default, the bending angle of water is chosen. Once all the fragment are attached, the total oxydation degree is computed. Since `neutral_reach` is set to "full", no fragement will be removed. On the contrary, in the example below `neutral_reach` is set to "random". The inputfile list is a single file which is a Cd₆₈Te₅₅ XYZ structure. The selected passivation scheme consists of satisfying all dangling bonds of surface Te atoms. Since there are already more cationic species than anionic species in the structure, adding only hydrogen will not allow to reach neutrality. Once all hydrogen are added, they will eventually be removed.

```

In[1]:
from src.nanocraft.pssv import passivate

output_dir = "./tmp"

settings = {
    'xyz_file': "Cd68Te55.xyz",
    'passiv_scheme': [['Te', 'H']],
    'binding_site': 'ontop',
    'facet_constrain': 'all',
    'coord_constrain': 'all',
    'neutral_reach': 'random'
}

settings_grid = grid(settings)

for settings in tqdm(settings_grid, desc="Progress"):
    passivate(settings=settings, output_dir=output_dir)

```

A valid input for a passivation which allows to reach zero total oxidation is given by the following settings:

```

In[1]:
settings = {
    'xyz_file': "Cd68Te55.xyz",
    'passiv_scheme': [['Cd', 'SH'], ('Te', 'H')], # not the same as [('Cd', 'SH'), ('Te', 'H')] !!
    'binding_site': 'ontop',
    'facet_constrain': 'all',
    'coord_constrain': 'all',
    'neutral_reach': 'pseudorandom'
}

```

Since the `binding_site`, `facet_contrain`, `coord.contrain` arguments are singleton of size 1, their value will be applied to each step of the passivation scheme which are namely: 1) saturate all surface Cd with S-H groups and 2) saturate all the surface Te with hydrogen atom. The "pseudorandom" keyword will have the effect of causing the removal of a few hydrogen on the surface Te, but most will remain. In priority, they will be removed from the Te with only one dangling bond. A zero oxidation degree can be reached.

An alternative passivation scheme could be obtained with the following settings:

```

In[1]:
settings = {

```

```

'xyz_file': "Cd68Te55.xyz",
'passiv_scheme': [[('Cd', 'SH'), ('Cd', 'SH'), ('Te', 'H')]],
'binding_site': [['bridge', 'ontop', 'ontop']]
'facet_constrain': [['100', 'all', 'all']]
'coord_constrain': 'all', # equivalent to [['all', 'all', 'all']]
'neutral_reach': 'pseudorandom'
}

```

Here, all cadmium atoms forming a bridge on 100 facet are first passivated. Then, all remaining cadmium are passivated with S-H group on top. Finally hydrogen are added on top of all tellurium atoms. Atoms/fragments are then removed to reach neutrality as already described.

3.6 Functionalization

3.6.1 Functionalization of capped NPs

The `fnctn` module handles the functionalization of already passivated NC surfaces by substituting selected capping ligands with user-provided ligands. It supports control over the number and placement of substitutions via density parameters, anchor fragment specification, and placement strategies like random or distance-based in order to minimize sterical hindrance.

All functionalization is controlled by the `functionalize` function, configured through a settings dictionary. The function substitutes existing ligands matching the specified anchor fragment with the new ligand from the provided XYZ file, optionally randomizing orientations. It is worth noting that this procedure requires a pre-passivated NC with ligands (a passivation fragment or a bigger ligand); it substitutes existing ones rather than adding to bare surfaces. It is not applicable to bare NPs; for those, use `functionalize_bare` instead (see next section). The number of substitutions can be absolute, percentage-based, or surface density (per nm²), capped at available sites.

`functionalize(settings, output_dir=None)`

ligand_file: str, required.

Path to the XYZ file of the ligand to attach. It must be pre-processed according to the alignment procedure discussed in Sect. 3.7.

nanocrystal_file: str, required.

Input passivated nanocrystal XYZ path.

density_type: str, required.

Type of density control: "**absolute**" (fixed number of ligands to be substituted), "**percentage**" (fraction of available ligands), or "**per_nm2**" (ligands per nm² surface area). For the latter, a tessellation method is used to estimate the surface area of the NC. The area of the detected bare cluster is computed.

density_value: int or float, required.

Density ($\sim$ number / fraction) of ligands that will be substituted. The expected value is an integer if **density_type**: '**absolute**', or a float otherwise. If requesting more substitutions than available anchors, it defaults to the maximum.

anchor_fragment: str, required.

Formula of the existing ligand fragment to substitute; must be present in the NC, write the anchor atom first.

placement_method: str, required.

Selection strategy of the ligands to be replaced. Choose "**random**" to select from a uniform distribution or "**distance**" (the first ligand is choosed randomly; the following are chosen so that the distance between all ligands is maximal.).

random_angle: bool, optional (default True).

Randomize ligand orientation around attachment axis. If true, each new ligands will have a random orientation.

random_seed: bool or int, optional (default True).

Enable randomness; provide seed for reproducibility. Be careful: if you fix the seed and use the `duplicate` functionality of the settings grid, all structure will be identical. In that specific case, you may want to either set the random seed to True, or remove the duplicate keyword and provide a list of integer as a seed instead. Like so, all structures will be different with controlled seed.

A typical call to the functionalization function is done as follows:

```

In[1]:
from src.nanocraft.fnctn import functionalize

output_dir = "./tmp"
nrep = 5

settings = {
    'ligand_file': "/home/ligands/PEG2.xyz",
    'nanocrystal_file': ["/home/test/Au236.xyz"],
    'density_type': "per_nm2",
    'density_value': 1.0, # 1.0 ligands per nm2 will be added (if possible)
    'anchor_fragment': "SiO2",
    'placement_method': "distance",
    'random_angle': True,
    'random_seed': True
}

settings_grid = grid(settings, duplicate=nrep)

for settings in tqdm(settings_grid, desc="Progress"):
    functionalize(settings=settings, output_dir=output_dir)

```

In this example, an Au NP consisting of 236 atoms is functionalized with PEG molecule. The target density is 1 ligand per nm². This density will only be reached if the number of fragment to replace, here SiO₂ is sufficient. The PEG will be placed so that they are most distance from each other and the initial choice for the placement of the PEG ligand is random amongst the pool of SiO₂. Each new ligand will be rotated by a random angle around in internal axis. The ligands has to be aligned using the pre-processing alignment functions.

3.6.2 Functionalization of bare structures

The `functionalize_bare` function, also part of the `fnctn` module, handles the functionalization of bare NP surfaces. It is applicable to any bare NP, from hollow cages to NCs. It works by attaching ligands directly onto surface anchor sites, without requiring any pre-existing capping layer. Unlike `functionalize`, which substitutes existing ligands, this function places ligands directly on bare surface sites identified through an alpha-shape tessellation of the surface. If the NC provided is not bare, the function will raise an error and suggest using `functionalize` instead.

The anchor sites are detected from the tessellation simplices based on the specified atom type (`anchor_type`) and binding mode (`anchor_site`), which controls whether the attachment is on-top (1 atom), bridge (2 atoms), or hollow (3 atoms). For each selected site, a dummy atom is placed at the computed binding position, accounting for a vertical offset derived from standard bond lengths, and subsequently replaced by the ligand following the same local frame substitution procedure as in `functionalize`.

`functionalize_bare(settings, output_dir=None)`

ligand_file: str, required.

Path to the XYZ file of the ligand to attach. It must be pre-processed according to the alignment procedure discussed in Sect. 3.7.

nanocrystal_file: str, required.

Input bare nanocrystal XYZ path. The nanocrystal must not contain any capping ligands.

anchor_type: str, required.

Atomic species of the surface atom used as anchor for ligand attachment.

anchor_site: int, required.

Binding mode, defined by the number of anchor atoms involved: 1 for ontop, 2 for bridge, and 3 for hollow sites.

density_type: str, required.

Type of density control: "absolute" (fixed number of ligands), "percentage" (fraction of available sites), or "per_nm2" (ligands per nm² of surface area). For the latter, the surface area is estimated from the alpha-shape tessellation.

density_value: int or float, required.

Target number or density of ligands to attach. The expected value is an integer if `density_type="absolute"`, or a float otherwise. If requesting more ligands than available sites, it defaults to the maximum.

`placement_method`: str, required.

Selection strategy for the anchor sites. Choose "random" to select from a uniform distribution or "distance" to maximize the spatial separation between placed ligands (the first site is chosen randomly; subsequent ones are chosen to maximize the minimum distance to all previously selected sites).

`random_angle`: bool, optional (default True).

Randomize ligand orientation around the attachment axis. If True, each ligand will be assigned a random rotation around its internal axis.

`random_seed`: bool or int, optional (default True).

Enable randomness; provide an integer seed for reproducibility. The same caution as in `functionalize` applies: if the seed is fixed and the `duplicate` functionality is used, all generated structures will be identical.

`nloc_COM`: int, optional (default 15% of total atoms).

Number of neighboring atoms used to compute the local center of mass at each anchor site. This local center of mass is used to determine the outward normal direction for ligand placement. Defaults to 15% of the total number of atoms in the NC. Reduce this value for highly heterogeneous systems.

`alpha_tessel`: float, optional (default 1.2).

Alpha parameter controlling the tightness of the alpha-shape tessellation used for surface site detection.

A typical call to the `functionalize.bare` function is done as follows:

```
In[1]:
from src.nanocraft.fnctn import functionalize_bare

output_dir = "./tmp"
nrep = 5

settings = {
    'ligand_file': "/home/ligands/PEG2.xyz",
    'nanocrystal_file': ["/home/test/Au236_bare.xyz"],
    'density_type': "per_nm2",
    'density_value': 1.0, # 1.0 ligands per nm2 will be added (if possible)
    'anchor_type': "Au",
    'anchor_site': 1, # ontop binding
    'placement_method': "distance",
    'random_angle': True,
    'random_seed': True
}

settings_grid = grid(settings, duplicate=nrep)

for settings in tqdm(settings_grid, desc="Progress"):
    functionalize_bare(settings=settings, output_dir=output_dir)
```

In this example, a bare Au NP consisting of 236 atoms is functionalized with a PEG molecule. The target density is 1 ligand per nm², which will only be reached if a sufficient number of on-top Au surface sites are detected by the tessellation. Ligands are placed so as to maximize their mutual distance, with the initial site chosen randomly from the available pool. Each ligand is rotated by a random angle around its internal axis. As with `functionalize`, the ligand must be pre-aligned using the pre-processing alignment functions described in the next section.

3.7 Basic molecular editing

The `molutils` module provides a set of molecular and atomic manipulation utilities applicable to any structure stored as a NumPy array, whether it is a full NP, a molecular fragment, or a ligand. Each row of such an array is formatted as [label, x, y, z]. The available functionalities span geometric operations, coordination filtering, and atom addition or removal. While the module is primarily intended for internal use, its functions can nonetheless be used directly to add atoms to a structure or remove them based on a coordination criterion.

Two functions from this module are of particular relevance for the functionalization workflow: `align_zaxis` and `align_zaxis_xyplane`. Both rotate a ligand molecule so that its main axis, defined by two user-specified atom types `zat1` and `zat2`, aligns with the z-axis, preparing it for attachment to NC surfaces. In practice,

the local frame of the ligand XYZ file is used as the local frame at the surface of the NP during substitution, making this pre-alignment step essential for correct ligand placement. In both functions, the alignment axis is defined by the farthest pair of atoms matching `zat1` and `zat2`, and `zat1` is placed at the origin by default (`zat1.origin=True`).

The two functions differ in how the ligand is oriented around the z-axis. `align_zaxis` performs only the axial alignment and leaves the azimuthal orientation arbitrary. `align_zaxis_xyplane` additionally rotates the ligand around z so that a specified third atom `xat1` lies in the xy-plane, with a preference for the positive x direction, ensuring a consistent and reproducible azimuthal alignment. Both functions return the rotated molecule as a NumPy array that can be directly saved and used as input for `functionalize` or `functionalize_bare`.

```
align_zaxis(mol, zat1, zat2, zat1_origin=True)
```

Aligns the `zat1-zat2` axis to the global z-axis using the farthest-spaced pair of matching atoms. No further rotation is applied. Returns the rotated molecule as a NumPy array (label, x, y, z).

```
align_zaxis_xyplane(mol, zat1, zat2, xat1, xat1_force_sel=None, zat1_origin=True)
```

Aligns the `zat1-zat2` axis to z, then rotates around z so that the closest `xat1` atom to `zat1` projects into the xy-plane. Use `xat1_force_sel=k` (0-indexed) to select the k-th atom of type `xat1` instead of the closest one. Returns the aligned molecule as a NumPy array.

A typical preprocessing workflow for a thiol ligand (propanethiol) using `align_zaxis_xyplane` is shown below. In this example, the hydrogen atom covalently bonded to the sulfur atom has been removed, as the ligand will be substituted onto an existing S-H fragment.

```
In [1]:
from src.nanocraft.molutils import align_zaxis_xyplane
from src.nanocraft.ioxyz import load_xyz, save_xyz

inputfile = "/home/propanethiol.xyz"
outputfile = inputfile.split(".xyz")[0] + "_aligned.xyz"

ligand = load_xyz(inputfile, sanity_check=False, center_COM=False)
ligand = align_zaxis_xyplane(mol=ligand, zat1='S', zat2='C', xat1='C')

save_xyz(ligand, outputfile)
```

In this example, the S-C axis (S being `zat1`, the attachment atom) is aligned to the z-axis using the farthest S-C pair; the closest C to S (`xat1`) is then rotated into the xy-plane. To use `align_zaxis` instead, simply omit `xat1` and call `align_zaxis(ligand, 'S', 'C')`. For ligands containing multiple atoms of the same type, `xat1_force_sel=1` (0-indexed) can be used to select the second atom of type `xat1`.

4 Visualization and diagnostics

Nanocraft provides a dedicated set of visualization tools gathered in the `src/viewer` folder. These tools are accessible through the `View` Jupyter Notebook. All visualization functions accept a list of XYZ filepaths as inputs.

4.1 Structure visualization

The `viz` module provides interactive 3D visualization of NP structures using the `py3Dmol` library. Multiple structures can be displayed simultaneously in a grid layout. The following functions are available.

```
visualize_xyz(inputfiles, **kwargs)
```

Visualize all structures provided in a list. Applicable to any type of NP.

```
visualize_nc_core(inputfiles, **kwargs)
```

Visualize the bulk core of the provided structures, as detected from `coordination.json`. Applicable to any type of NP.

```
visualize_nc_surf(inputfiles, **kwargs)
```

Visualize the outermost layer of the bulk core. Applicable to any type of NP.

```
visualize_nc_ligands(inputfiles, **kwargs)
```

Visualize all detected ligand atoms of the provided structures. Applicable to any NP with a capping layer.

```
visualize_nc_all(inputfiles, **kwargs)
```

Calls `visualize_nc_core`, `visualize_nc_surf`, and `visualize_nc_ligands` side by side for a given structure, enabling a quick layered inspection. Applicable to any type of NP.

```
visualize_local_frame(cluster, r0, ex, ey, ez, scale=1.5)
```

Visualize a structure loaded with `load_xyz` with a local coordinate frame overlaid, defined by an origin `r0` and three unit vectors `ex`, `ey`, `ez`. Applicable to any type of NP.

All functions share a common set of optional keyword arguments to control the rendering: `ncols` (number of columns in the grid layout), `style` (rendering style: "default", "sphere", "stick", "line", "cross"), `color_scheme` (e.g., "Jmol"), `width`, `height`, `background`, and `spin` (enable rotation animation).

4.2 Facet visualization

The `vizfct` module provides visualization of atoms belonging to specific facets, edges, or vertices of a NC. It is applicable to bare NCs only. Atoms belonging to the selected facet(s) are highlighted using a dummy atomic species.

```
visualize_facets(inputfiles, facet_or_family='all', intersection=False, dummy='U', **kwargs)
```

Highlight atoms belonging to the specified facets or family of facets. `facet_or_family` accepts either a list of Miller index triplets (e.g., `[[1,0,0], [-1,0,0], [1,1,-1]]`) or a list of family strings (e.g., `['100']`). When `intersection=True`, only atoms belonging to all specified facets simultaneously are highlighted; otherwise, the union is shown. The `dummy` keyword sets the chemical label (default 'U') temporarily assigned to matching surface atoms to visually distinguish them in the 3D rendering.

4.3 Tessellation visualization

The `viztssl` module provides 3D visualization of the surface tessellation of NP structures using the `plotly` library. It is applicable to bare NCs only and is particularly useful for inspecting and tuning the `alpha` parameter of the alpha-shape tessellation.

```
visualize_simplices(inputfiles, method='hull', alpha=1.2, **kwargs)
```

Visualize all triangular simplices generated by the tessellation algorithm. Useful for verifying that the tessellation correctly captures the surface of the structure.

```
visualize_patches(inputfiles, method='hull', alpha=1.2, threshold=10.0, **kwargs)
```

Visualize the surface as higher-order polygonal patches, obtained by merging coplanar simplices whose normals deviate by less than `threshold` degrees. This corresponds to the highest-order enclosing polygon representation used for shape identification (see Sect. 3.1).

Both functions share the same optional keyword arguments as the `viz` module for layout control (`ncols`, `width`, `height`, `background`), as well as a `style` argument accepting "default", "solid", "wireframe", or "points".